



POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE

EXECUTIVE SUMMARY OF THE THESIS

Robot air hockey via hierarchical reinforcement learning

LAUREA MAGISTRALE IN COMPUTER SCIENCE AND ENGINEERING - INGEGNERIA INFORMATICA

Author: THOMAS JEAN BERNARD BONENFANT

Advisor: PROF. MARCELLO RESTELLI

Co-advisors: AMARILDO LIKMETA DAVIDE SALAORNI

Academic year: 2023-2024

1. Introduction

Reinforcement learning enables learning from trial and error by interacting with the environment, which provides states and rewards. However, applying this trial-and-error approach can be problematic in some applications. For example, in real-world robotic control, this approach can lead to safety issues and unwanted damage to the robotic system and its environment. Therefore, reinforcement learning is often first applied in a simulated environment before being deployed in the real world.

The goal of this work is to develop an agent capable of playing entire air hockey games while adhering to all operational constraints. Specifically, the agent must control a general-purpose manipulator, achieving the best possible performance in terms both of offense and defense.

2. Related Works

A previous work based on trajectory optimization [2] with a 7 DoF manipulator was conducted for the Air hockey hit task.

To achieve safe exploration ATACOM [3] can be used. It transforms a constrained RL problem into an unconstrained RL problem. This allows to use any model-free reinforcement learning algorithm.

3. Preliminaries

A Finite Markov Decision Process (MDP) is a tuple $\mathcal{M} = (\mathcal{S}, \mathcal{A}, p, r, \gamma, \mu)$ specified by a finite set $\mathcal{S}$ of possible states, a finite set $\mathcal{A}$ of actions, a transition probability function $p : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \rightarrow [0, 1]$, a reward function $r : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$, a discount factor $\gamma \in (0, 1)$, and a probability distribution over the initial states $\mu : \mathcal{S} \rightarrow [0, 1]$. The goal of reinforcement learning is to maximize the expected discounted return defined by

4. Air hockey competition

The Air hockey competition was organized to close the sim-to-real gap, i.e., the differences and challenges that arise when transferring a robotic system trained in a simulated environment to the real world.

It consists in three simulated stages: *Warm Up*, *Qualifying* and *Tournament*. Each stage consists of different tasks required for robot air hockey.

In the Warm-Up stage the participants were given an ideal environment with a 3 DoF planar robot and two tasks: *hit* and *defend*. In the first one the objective is to hit the puck and score a goal. In the other the objective is to defend the goal from an incoming puck.

In the Qualifying stage, the participants were

given an environment exposing an interface to control a general-purpose robot, the KUKA-iiwa14 LBR which has 7 DoFs. This stage includes also the *prepare* task: the objective is to move the puck away from the table borders for facilitating the *hit* task.

In the tournament stage, the access to a baseline opponent was given and matches against other participants were organized.

5. Methodology

5.1. State space

The observations provided by the original environment were slightly modified. Indeed the angular position θ and velocity $\dot{\theta}$ of the puck was discarded as well as the z-axis element of the puck's position p_{puck} and velocity $\dot{p}_{puck}$. The end-effector's position p_{ee} and velocity $\dot{p}_{ee}$ were computed using the forward kinematics of the robot:

$$\begin{aligned} p_{ee} &= FK(q) \\ \dot{p}_{ee} &= J(q)\dot{q} \end{aligned}$$

where $q = \{q_i | i \in 1, \dots, 7, q_i^{\min} \leq q_i \leq q_i^{\max}\}$ fully represents the joint angles of the robot, FK is the forward kinematics function and J is the geometric Jacobian matrix of the forward kinematics function. Also the z-axis component of the position and velocity of the end-effector were discarded. The resulting state space is given by

$$s = [p_{puck} \ \dot{p}_{puck} \ q \ \dot{q} \ p_{ee} \ \dot{p}_{ee}]$$

with $s \in \mathbb{R}^{22}$ for the 7 DoF robot and $s \in \mathbb{R}^{17}$ for the 3 DoF robot. Finally, the state space was scaled between -1 and 1:

$$s_{scaled} = \frac{2(s - s_{low})}{s_{high} - s_{low}} - 1$$

where $s_{low} \leq s \forall s$ and $s_{high} \geq s \forall s$.

5.2. Action space

Two different approaches were used for modeling the action space.

Task space control: A high-level control setting where the agent controls the end-effector's position. This is achievable using the inverse kinematics of the robot to find a configuration of the robot that places the end-effector in the wanted position.

$$\begin{aligned} \bar{q}_t &= IK(\bar{p}_{ee,t}) \\ \dot{\bar{q}}_t &= \frac{\bar{q}_t - q_{t-1}}{\delta t} \end{aligned}$$

where $\bar{q}_t$ is the desired joint configuration computed using inverse kinematics from the desired end-effector's position $p_{ee,t}$ at time t and $\dot{\bar{q}}_t$ is the desired joint velocity computed using finite differences.

The computed joint velocities were next optimized using *weighted null space optimization*, described in [2], that takes into account the joint velocity limits.

Joint space control: The agent directly controls the joint accelerations $\ddot{q}$. This control variable is then converted to joint position and velocities using ATACOM and submitted as a reference variable to the underlying controller. The ATACOM transformation produces a command [joint_positions, joint_velocities] that reduces greatly the number of violated constraints excluding the joint velocity constraint. Reducing the joint acceleration limits solved the joint velocity constraints problem. The action space too was scaled between -1 and 1.

5.3. Reward function

The designed reward is task-dependent, given that a policy is trained for each task. It is also shaped as the success reward of each task is very sparse. Generally the reward function introduces a penalty for each violated constraint proportional to its penalty points:

$$r(s, a) = r_t(s, a) + r_c(s, a),$$

where $r_t(s, a)$ is the task-dependent reward and $r_c(s, a)$ is the penalty for violating the constraints.

$$r_c(s, a) = \frac{\sum_i \left(\phi_i \cdot \frac{1}{N} \sum_{j=1}^N \sigma_j \right)}{\sum_i \phi_i},$$

where N is the number of violated constraints, ϕ_i is the penalty points associated to the violation of constraint i and σ_i is the *slack*, i.e. the difference between the current value of the constraint variable and its limit.

Additional tasks other than the ones specified by the competition were designed for learning behaviors useful for a complete game. A *return home* task in which the robot needs to achieve

a certain configuration and remain still, and a *counter-attack* similar to the *defend* task but gives the robot the possibility to score a goal while defending from the incoming puck.

The reward function $r_t(s, a)$ for each task is now described:

Defend and counter-attack: Two agents for the defend and counter-attack tasks. In these tasks the episode starts with the puck moving towards our goal. In both cases, the agent receives a high penalty (-100) when the puck breaches our goal. When the agent successfully interacts with the puck, a reward (+10) is granted, coupled with reward proportional to the norm of the puck's velocity, with a negative sign while defending and positive sign while counter-attacking. This incentivizes the agent to interact with the puck and lower its velocity while also protecting the goal in the defend task, while pushing back the puck in the counter-attack. Moreover, in the counter-attack, the agent can also score in the opponent's goal, for which he receives an additional large bonus of 1000.

$$r = \begin{cases} 1000 & \text{if scored} \\ 10 + A \cdot \|\dot{p}_{puck}\| & \text{if interacted with puck} \\ -100 & \text{if opponent scored a goal} \end{cases}$$

where A is a constant term with positive sign in the counter-attack task, negative in the defend task.

Hit: The reward function of the hit task is based on the construction of a triangle with the opponent's area and the puck as vertices (Figure 1). If the puck, after the hit, lies inside the triangle, a reward is assigned, proportional to the puck's velocity; if the puck is outside the triangle, a penalty is assigned. The more the puck is outside the polygon, the bigger the penalty.

$$r_{\text{before hit}} = \begin{cases} -\frac{\|p_{ee} - p_{puck}\|}{0.5 \cdot \text{table_diag}} & \text{if no hit,} \\ 100 + 10 \frac{(1 - (\frac{2\alpha}{\pi})^2) \|\dot{p}_{ee}\|}{\text{max_vel}} & \text{if hit the puck,} \\ \frac{1}{1 - \gamma} & \text{if scored} \end{cases}$$

where table_diag is the diagonal of the table and max_vel is a constant equal to the maximum velocity observable in the environment, while $\alpha = \arctan2(p_{ee,y}, p_{ee,x})$.

If the agent hits the puck, it receives an instantaneous reward and, after that, the triangle approach is applied.

$$r_{\text{after hit}} = \begin{cases} 10 + \|\dot{p}_{puck}\| & \text{if puck inside triangle,} \\ -\text{diff_angle} & \text{if otherwise} \end{cases}$$

$$\text{diff_angle} = \arctan2(\dot{p}_{puck,y}, \dot{p}_{puck,x}) - \text{angle_border}$$

where angle_border is the angle between the puck velocity vector and the closest triangle border.

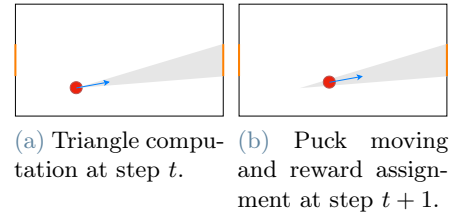


Figure 1: Triangle construction for assigning reward after the mallet hit the puck.

Prepare: The reward function for the prepare task is a sparse reward given at the last step with a high reward (1000) if the task is successful, otherwise -100.

Return home: At each step of the training process, the agent incurs a penalty based on the distance to the *home configuration*

$$r = -\|q - q_{home}\|$$

where q_{home} is the initial position of the robot before a game starts or after a reset. For this task, the robot was initialized to a configuration drawn from the distribution of terminal states of the other tasks. This distribution is necessary for creating experiments with different initial positions, ensuring that the agent can learn to reach q_{home} independently of its starting position.

5.4. SAC Agent

SAC [1] was used to implement different agents during the Robot air hockey competition. As illustrated in Figure 2 the agent was composed of two components: the policy's neural network and the ATACOM transformation.

Neural Network: The trained neural network is a two layer feed forward neural network with 256 neurons per hidden layer and an output layer with $2 \times n$ neurons where n is the number of controlled joint accelerations. The input layer depends on the state space of the task’s environment. The nonlinearity of the neural network comes from the rectified linear unit (ReLU) activation function.

To achieve exploration with a continuous action space the outputs of the neural network are used as mean and standard deviation of a gaussian distribution. Finally a *tanh* transformation is applied to the action sampled from the gaussian distribution. This is done to ensure that the action is bounded between -1 and 1. The final action is interpreted as the desired joint accelerations $\ddot{q}$.

ATACOM Transformation: This component receives as inputs the desired joint accelerations $\ddot{q}$ computed from the preceding component and outputs joint positions and velocities $[q \ \dot{q}]$ that don’t break constraints using the ATACOM algorithm. However, ATACOM was unable to adhere to the constraints on the joint velocity. This is why we also limited the maximum joint accelerations between -1 and 1.

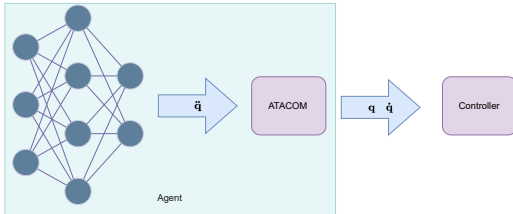


Figure 2: SAC Agent with ATACOM.

The SAC Agent was used for the *defend*, *counter-attack* and *home* tasks defined in section with their reward functions.

5.4.1 Defend and Counter-attack tasks

For the *defend* and *counter-attack* task a curriculum learning (CL) [4] approach was used to gradually increase the difficulty of the task and facilitate training. Manual curricula across three levels of complexity, easy, medium and hard, were devised. These levels differ in the initial configuration of the puck, considering its position and velocity.

1. **Easy:** In the easy level, the puck’s velocity was constrained along the y-axis and kept

the velocity in the x-axis low. Additionally, we positioned the puck at the end of the table, allowing the agent more reaction time.

2. **Hard:** Both the x-axis and y-axis puck’s velocities were increased, and its initial position was set in the middle of the table.
3. **Medium:** The medium task served as an intermediary between the hard and easy tasks.

The agent was trained for a total of 8000 epochs, where each epoch consists in 30 episodes of experience. In the first 3000 epochs the agent was trained with the easy level. Next a *soft* switch to the *medium* level task for 1000 epochs increasing the proportion of episodes drawn from the *medium* level task. Between epoch 4000 and 6000 the agent is trained exclusively on the *medium* level task. As before, a *soft* switch to the *hard* task is performed and finally train exclusively on the *hard* level task for the last 1000 epochs.

5.4.2 Home task

In this additional task the goal is to train an agent able to return to the home position from any arbitrary position. The initial joint positions for the robot were drawn from a previously collected dataset of configurations. In this task the observation space was reduced to only include the joint positions and velocities:

$$s = [q \ \dot{q}]$$

The reward function is described in 5.3. No curriculum was needed as the *home* task is simple.

5.5. RL Optimized rule-based agent

For the *hit* and *prepare* task defined in 5.3 a rule-based controller was implemented. This controller was then optimized through PGPE [5]. In this case the robot is controlled using the *task space*, combining inverse kinematics and Anchored Quadratic Programming (AQP) [2]. The policy set was divided into 2 main branches that resemble the challenge tasks: hit and prepare. Each policy is further divided into phases, common to both of them:

- *Adjustment:* move the end-effector on (i) the line linking the puck and the goal for the hit; (ii) the vertical line parallel to the short side, passing through the puck, for the prepare.

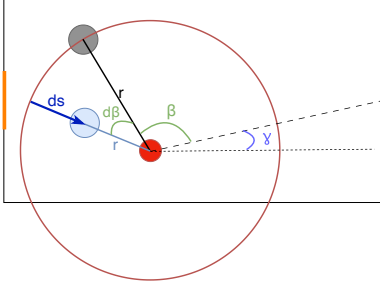


Figure 3: Polar coordinates used in the Rule-based controllers

- *Acceleration*: accelerate the end-effector and hit the puck. The hit policy hits the puck to score a goal, while the prepare one makes a more soft bump, in order to re-adjust the puck's position;
- *Final*: this phase only applies to the hit policy. Here, the agent keeps hitting the puck making it reach a desired acceleration, then slowly stops the end-effector following a curved trajectory.

In particular, the hit policy computes two quantities, $d\beta$ and ds in the following way:

$$d\beta = \begin{cases} (\theta_0 + \theta_1 \cdot t_{\text{phase}} \cdot dt) \cdot \text{correction} & \text{adjustment phase} \\ \frac{\text{correction}}{2} & \text{acceleration phase} \\ (\theta_0 + \theta_1 \cdot dt) \cdot \text{correction} & \text{slow-down phase} \end{cases}$$

$$ds = \begin{cases} \theta_2 & \text{adjustment phase} \\ \frac{ds_{t-1} + \theta_3 \cdot t_{\text{phase}} \cdot dt}{\text{radius} + r_{\text{mallet}}} & \text{acceleration phase} \\ \text{constant} & \text{slow-down phase} \end{cases}$$

where *radius* is the distance between the center of the puck and the end-effector, while *correction* is always defined as:

$$\text{correction} = \begin{cases} \beta - 90 & y_{\text{puck}} \leq 0 \\ 270 - \beta & y_{\text{puck}} > 0 \end{cases}$$

All the quantities appearing in the computation of $d\beta$ and ds are explained graphically in Figure 3

5.6. Hierarchical agent

For the tournament stage of the competition, an agent capable of playing entire matches is needed. Having previously implemented different agents for each task we can play an entire match alternating between these specific agents. The difficulty now is to define how to switch between these subpolicies while being competitive against an opponent.

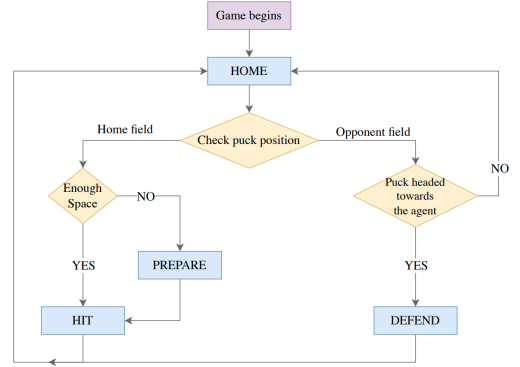


Figure 4: Switcher structure.

To do so, a high-level agent has been developed. This agent decides what policy to execute, based on the observation of the environment and on two main components: the Switcher and the Finite State Machine.

The switcher selects a new policy once the current one has met a termination condition and informed the high level agent. This new selected policy is selected according to a set of rules applied on the observation state. The switcher's logic is illustrated in Figure 4. The selected policy could potentially be the previous one. This can happen, for example, if the agent performed a prepare but the puck is still in an unsuitable position for hitting, a second prepare might be necessary.

The switcher's rules relies on some parameters, therefore it could be trained formulating an optimization problem to find an optimal set of parameters. To do this we can use PGPE [5].

Policy Gradients with Parameter-Based Exploration (PGPE) is a reinforcement learning algorithm that enhances exploration by directly perturbing the parameters of the policy. Instead of adding noise to the actions at each step, PGPE samples the policy parameters from a distribution, leading to a deterministic execution of the policy based on these sampled parameters. This method is particularly advantageous for optimizing non-differentiable policies, such as the switcher implemented in this work. Using PGPE, a gaussian distribution ρ over the switcher's parameters θ . The algorithm updates the mean μ and standard deviation σ of ρ , allowing effective exploration and optimization. For further details refer to [5].

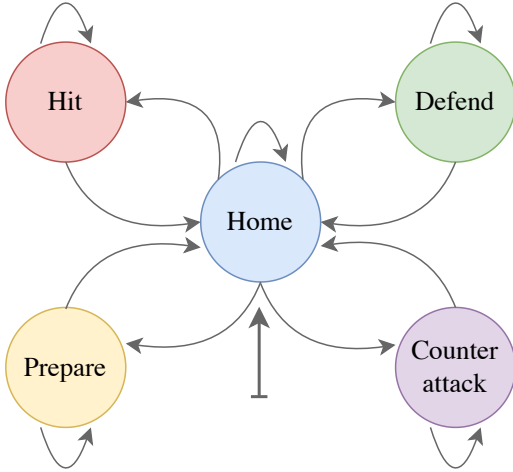


Figure 5: The Finite State Machine.

5.7. Noise Filtering

During the challenge our agents were evaluated on a remote simulator that was slightly different from the local environments we used. In order to mimic a real-world scenario this remote simulator introduced disturbances that could create problems to our agents.

The noise on the puck observations and the loss of tracking were particularly problematic. To solve this problem a kalman filter [6] was used on the puck’s position to filter the noise. Additionally, the noisy observations of joint positions and velocities were ignored and the reference joint positions and velocities given to the underlying controller at the previous step were used as observations instead.

6. Experimental Results

6.1. Warm up phase

In the Warm up phase only the *hit* and *defend* agents were evaluated. At that time these two agents were trained with SAC using a *Task space control* strategy. The inverse kinematics approach was employed to compute the joint positions from the desired end-effector position. The results obtained were good in term of success rate but the agent was classified as *non deployable* as the penalty points derived from constraint violations were too high (> 1500):

Hit success	Defend success	Max Penalty points
85.0%	12.2%	4402.0

Table 1: Warm up 3DoF planar robot, server evaluation

6.2. Qualifying phase

During the qualifying phase the participants were given access to the environment with the 7 DoF robot and an additional task, the *prepare* task. In this environment, the tasks became much harder to solve while adhering to constraints as the end-effector was now capable to move in all directions given the high number of DoF. During this phase the *hit* and *defend* agent were implemented using a Rule-Based controller similar to the one described in ?? with a sparser reward function. Still, the hit task was the most problematic in terms of constraints, and this contributed to having our agents classified as *improvable*.

The Defend agent was implemented using SAC as described in 5.4 using a *Joint space control* strategy coupled with ATACOM. ATACOM proved to work very well in this setting achieving a very low number of constraint violations.

The success rates and penalty points are illustrated in tables 2 and 3.

Hit success	Defend success	Prepare success	Max Penalty points
22.7 %	61.6 %	36.2 %	920.0

Table 2: Qualifying success 7DoF robot, server evaluation

Hit penalty	Defend penalty	Prepare penalty
920.0	1.5	329.0

Table 3: Qualifying penalty points

6.3. Tournament

During the tournament phase an agent capable of playing entire matches was needed. Adhering to constraints was also crucial because violating too many constraints would result in losing the match regardless of the score.

The employed subpolicies are the ones described in 5: a Rule-based agent was employed for hitting and preparing the puck while a SAC agent

was employed to defend the goal hitting the incoming puck attempting to score the opponent's goal. This counterattack policy was used because of the low performance of the rule-based hit policy in the hitting task. It was trained on a modified *defend environment* where the reward function includes a positive reward if the agent scores a goal as described in section 5.4.1.

The home policy was essential because switching policy too fast without first returning in the default position and low velocities resulted in a lot of violations. This also gave the counterattacking SAC policy an initial position it is familiar with.

The switcher policy therefore included four policies:

- hit rule-based
- counterattack SAC
- prepare rule-based
- return home SAC

For the competition, the parameters of the switcher were tuned manually. During the final phase of the competition our hierarchical agent achieved a winning rate of 50 % against the other participants always adhering to constraints.

7. Improving the challenge's solution

After the challenge, efforts were made to improve the solution. Specifically the objective was to have an agent capable of achieving a good success rate on the hit task and then optimize the switcher to find an optimal set of parameters.

7.1. Improving the Hit Task

To improve the Hit Task a SAC agent with ATACOM was used. However, our implementation of ATACOM had problems respecting the joint velocity constraints. These violations were previously indirectly solved limiting the joint accelerations given to ATACOM between -1 and 1. First, the last joint acceleration was removed from the action space, as this joint was only responsible for the rotation of the end-effector. From now on, the acceleration given to the last joint is assumed to be 0.

Next, with trial and error a new set of joint acceleration limits was found that kept the joint velocity constraints low:

$$|\ddot{q}_{limits}| = [6 \quad 11 \quad 5 \quad 6 \quad 1 \quad 9]$$

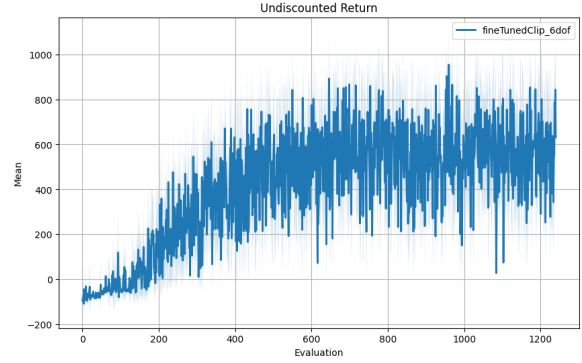


Figure 6: This figure represents the undiscounted mean return computed using the target policy of the agent. Each evaluation is performed every 20000 environment steps over 20 episodes.

The previous joint acceleration limits $-1 \leq \ddot{q}_i \leq 1$ were too restrictive, while this new set of joint accelerations limits enabled our agent to be more reactive.

The reward function used for this hit task is:

$$r = \begin{cases} \frac{\|p_{puck} - p_{ee}\|}{0.5 \text{ table_diag}} & \text{if no hit} \\ 50 \cdot \|\dot{p}_{puck}\| & \text{once after hit} \\ 1000 & \text{if success} \end{cases} \quad (1)$$

The training environment was modified in order to aggregate all the remaining steps from the moment the puck is hit into one. Specifically, after the puck is hit, a default policy is used to make the agent stop smoothly and the training environment submits to the training algorithm only the last step of the episode. This sped up the training as all the steps during which the puck was not controllable by the agent were ignored.

A plot of the undiscounted return is illustrated in Figure 6. In this figure each point corresponds to the undiscounted return computed from an evaluation of the target policy over 20 episodes every 20000 environment steps. The mean return reaches a value around 600. This value is not directly linked to the success rate as the reward function introduces also a positive reward proportional to the puck's velocity.

This new hit agent achieved on a local evaluation without noise a 58 % success rate over 100 episodes with a mean undiscounted return of 652, demonstrating an improvement over the previous rule-based hit policy.

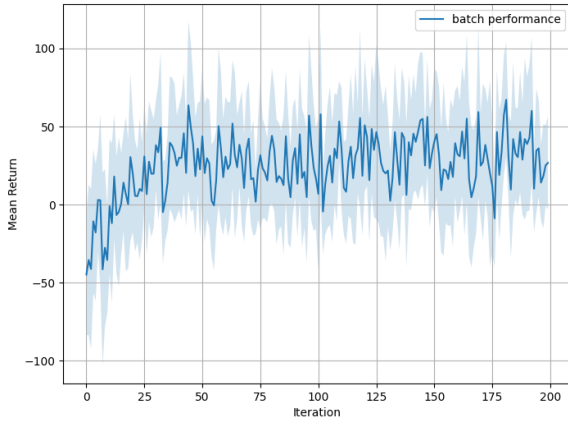


Figure 7: In this figure each point correspond to the mean return obtained from a distribution ρ of parameters. The return during training increases over zero, meaning that the hierarchical agent is consistently winning

7.2. Improving the Switcher

Other than replacing the rule based hit policy, a new home policy was implemented based on a PD controller:

$$\ddot{q} = K_p \cdot (q_{home} - q) - K_d \cdot (\dot{q})$$

with q_{home} equal to the desired position, to return to and $K_p = 10$ and $K_d = 1$. These values were found by hand.

The switcher is a non differentiable policy that depends on a parameter vector θ . It can be optimized with PGPE [5] and find an optimal set of parameters. The reward function is a simple sparse reward:

$$r = \begin{cases} 1000 & \text{if our agent scores a goal} \\ 333 & \text{if the opponent gets a fault} \\ -1000 & \text{if the opponent scores a goal} \\ -333 & \text{if our agent gets a fault} \end{cases}$$

In this training environment episodes last 3000 steps and the discount rate γ is set to 0.999. The opponent used in this environment is still the baseline provided by the competition’s organizers. A plot of the mean return of the parameters distribution is illustrated in Figure 7.

8. Conclusions

In this work, reinforcement learning was successfully applied to the air hockey task using a general-purpose manipulator. The agent was

able to play entire matches without violating too many constraints, thanks to the ATACOM algorithm. The SAC method outperformed the rule-based controller for the hitting task, achieving a higher success rate. The rule-based switcher effectively selected the appropriate policy at the right time, with its parameters slightly optimized via PGPE.

The developed rule-based switcher currently uses a few simple parametric rules. Future work could involve implementing a more complex switcher with additional rules that also take the opponent’s actions into account.

An other strategy could be to apply an end-to-end hierarchical reinforcement learning approach, such as HIRO [?], where a single low-level policy receives from the high-level policy a vector to determine the desired behavior.

Since air hockey is a multi-agent setting, investigating a self-play learning strategy could also be beneficial. This approach could help the agent improve by continuously adapting to and learning from interactions with itself or other agents.

References

- [1] Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. *CoRR*, abs/1801.01290, 2018.
- [2] Puze Liu, Davide Tateo, Haitham Bou-Ammar, and Jan Peters. Efficient and reactive planning for high speed robot air hockey. In *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 586–593, 2021.
- [3] Puze Liu, Davide Tateo, Haitham Bou-Ammar, and Jan Peters. Robot reinforcement learning on the constraint manifold. In Aleksandra Faust, David Hsu, and Gerhard Neumann, editors, *Proceedings of the 5th Conference on Robot Learning*, volume 164 of *Proceedings of Machine Learning Research*, pages 1357–1366. PMLR, 2022.
- [4] Sanmit Narvekar, Bei Peng, Matteo Leonetti, Jivko Sinapov, Matthew E. Taylor, and Peter Stone. Curriculum learning for reinforcement learning domains: A framework and survey, 2020.

- [5] Frank Sehnke, Christian Osendorfer, Thomas Rückstieß, Alex Graves, Jan Peters, and Jürgen Schmidhuber. Parameter-exploring policy gradients. *Neural Networks*, 23(4):551–559, 2010. The 18th International Conference on Artificial Neural Networks, ICANN 2008.
- [6] Greg Welch, Gary Bishop, et al. An introduction to the kalman filter. 1995.